

claude-windows / 45ea3a91-654d-4972-9cd3-65f35db54caf

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\45ea3a91-654d-4972-9cd3-65f35db54caf\subagents\workflows\wf_2a64bc88-2a8\agent-acb93ba37cbfac8cd.jsonl`  
- SHA-256: `6f5fcc6d41b5e9176371382020841f746e913687a54accf780fc501cf1f9995af`  
- Source modified: `2026-06-10T06:15:33+00:00`  
- Imported at: `2026-07-05T16:48:27+00:00`  
- project: `wf_2a64bc88-2a8`  
- session_id: `45ea3a91-654d-4972-9cd3-65f35db54caf`
```

Transcript

1. user (2026-06-10T06:12:46.615Z)

CONTEXT ? two sibling repos on a Windows machine:

- INDEXER: C:/Users/avidu/Projects/badminton-highlight-indexer ? local AI badminton(->racket-sports) highlight indexer + rally detector. FastAPI + SQLite + ffmpeg + GPU CV via WSL; Gemini = paid cloud AI. docs/ tree is rich. Current branch docs/next-steps-multisport (master = main branch).

- COLLECTOR: C:/Users/avidu/Projects/sports-data-collector ? golden-label acquisition/curation/ingestion CLI (system-of-record candidate for training corpus).

Project state (June 2026): scaffolding complete; owned-model roadmap agreed (docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md) but NO platform/owned-model implementation started; next committed platform slice = async job model. Licensing guardrail: MIT/Apache-2.0/BSD only for code+data+weights; paid LLMs are inference-only, never training-data sources. ENVIRONMENT RULES: this dev environment has NO GPU/torch/cv2 ? do NOT attempt to run the video pipeline or import torch/cv2 modules. Test suites are offline/fully-mocked and SAFE to run. You are READ-ONLY: never edit/create/delete repo files; running tests and git read commands is allowed.

QUALITY BAR: cite file paths (and line numbers where possible) for every finding. Report what IS in the code/docs, not what you assume. Be skeptical and concrete. Your final structured output is consumed programmatically by an orchestrator ? return raw data, not prose for a human.

ROLE: adversarial verifier. An auditor ("eng-pipeline") claimed the finding below. Try to REFUTE it against the actual files. Default to isReal=false if the evidence does not hold up, the behavior is documented/deliberate (e.g. listed in docs/CODE_MAP.md gotchas or DEFERRED_HARDENING_PLAN.md as a known deferral ? unless the auditor adds genuinely NEW consequence), or the severity is materially overstated.

FINDING: {"title":"Nested config sections silently drop unknown keys ? entire indexing.wasb block is unconfigurable; live knob yolo_prefetch_workers unsettable while dead knobs ship in config.json","severity":"high","area":"config system","file":"C:/Users/avidu/Projects/badminton-highlight-indexer/backend/config/models.py:46-74,109-112","evidence":"Only AppConfig sets model_config={'extra':'allow'} (models.py:109-112); nested IndexingConfig/ValidationConfig use Pydantic v2's default extra='ignore'. Verified empirically:

AppConfig.model_validate({'indexing': {'wasb': {...}, 'yolo_prefetch_workers': 0, 'debug_duration': 30}}) ? all three keys absent from cfg.get('indexing'). Yet wasb_runner.py:43-55 (WasbConfig.from_indexing_cfg) reads idx_cfg.get('wasb', {}) and its docstring says 'built from config.json -> indexing.wasb'; wasb_hybrid.py:89 reads idx_cfg.get('wasb', {}); yolo_hybrid.py:342 reads yolo_prefetch_workers (default 2); gemini.py:61 reads indexing.debug_duration. None of these keys exist in IndexingConfig, so any config.json value is silently discarded and the hardcoded defaults always win. Inversely, IndexingConfig declares yolo_parallel_chunks/yolo_chunk_workers (models.py:59-60, also in config.json:38-39 and README.md:134-135) which are read NOWHERE in the codebase.", "why_it_matters": "WASB is the live shuttle substrate; its weights path, conda env, repo dir, sport, and timeout cannot be changed through the documented config path ? a user pointing indexing.wasb.weights_path at new weights gets the default weights with zero warning (extra='allow' was meant to 'not break' configs but only works at top level). The model-vs-code key inversion (dead keys declared, live keys undeclared) shows the typed-config migration was never reconciled against actual .get() call

sites. NEW finding ? not in CODE_MAP gotchas (CODE_MAP even documents 'indexing.wasb.velocity_threshold(0.02)' as if it were configurable)."} Steps: open the cited file(s) and any counter-evidence files; verify quotes/line claims; judge severity honestly. Cheap greps allowed; do NOT run the pipeline. Return isReal, confidence, and a 1-3 sentence verdict (with corrected detail if partially right).

2. user (2026-06-10T06:12:56.958Z)

```
1      """Pydantic models for the application configuration.
2
3      This module defines the canonical, validated, typed configuration for the
4      Badminton Highlight Indexer. Built-in defaults live here (single source of truth).
5
6      The models support graceful migration from old config.json files via extra="allow".
7      """
8
9      from __future__ import annotations
10
11     from typing import Any, Literal
12
13     from pydantic import BaseModel, Field, field_validator
14
15     Sport = Literal["badminton", "tennis", "pickleball", "table_tennis", "cricket"]
16
17
18     class ValidationRelevanceConfig(BaseModel):
19         court_green_lower: list[int] = Field(default=[35, 40, 40])
20         court_green_upper: list[int] = Field(default=[85, 255, 255])
21         court_pixels_min_ratio: float = Field(default=0.05, ge=0.0, le=1.0)
22
23
24     class ValidationConfig(BaseModel):
25         min_resolution_width: int = 1280
26         min_resolution_height: int = 720
27         min_fps: float = 29.9
28         min_blur_threshold: float = 20.0
29         max_scene_cuts_per_minute: float = 0.5
30         relevance: ValidationRelevanceConfig =
31         Field(default_factory=ValidationRelevanceConfig)
32
33     class TrackNetConfig(BaseModel):
34         wsl_distro: str = "Ubuntu"
35         repo_dir: str = "~/models/TrackNetV4"
36         conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
37         conda_env: str = "TrackNetV4"
38         weights_path: str = ""
39         queue_length: int = 5
40         stage_in_wsl: bool = True
41         wsl_stage_dir: str = "~/clips"
42         timeout_sec: int = 0
43         velocity_threshold: float = 0.02
44
45
46     class IndexingConfig(BaseModel):
47         default_segmenter: str = "yolo_hybrid"
48         use_gpu: bool = True
49         motion_threshold: float = 0.08
50         min_segment_duration: float = 3.0
51         dead_time_merge_gap: float = 2.0
52         chunk_duration: float = 90.0
53         chunk_overlap: float = 10.0
54         detector_model: str = "fasterrcnn_resnet50_fpn"
55         detector_score_threshold: float = 0.5
56         yolo_velocity_threshold: float = 0.15
```

```

57     yolo_frame_skip: int = 3
58     yolo_tracker: str = "bytetrack.yaml"
59     yolo_parallel_chunks: bool = False
60     yolo_chunk_workers: int = 2
61     min_action_window_duration: float = 2.0
62     log_yolo_candidates: bool = True
63     store_yolo_candidates: bool = True
64     ai_handoff_padding: float = 2.0
65     ai_max_workers: int = 5
66
67     tracknet: TrackNetConfig = Field(default_factory=TrackNetConfig)
68
69     @field_validator("default_segmenter")
70     @classmethod
71     def segmenter_must_be_registered(cls, v: str) -> str:
72         # Registry check happens at runtime in main/segmenter selection.
73         # Here we just allow any string for forward compatibility.
74         return v
75
76
77     class OutputConfig(BaseModel):
78         default_highlights_name: str = "highlights.mp4"
79         db_name: str = "sports_indexer.db"
80         # Directory where the DB, highlight reels, and processed clips are written.
81         # The CLI's --output-dir overrides this at startup (see backend/main.py::main).
82         output_dir: str = "output"
83
84
85     class StitchingConfig(BaseModel):
86         begin_padding: float = 0.5
87         end_padding: float = 0.5
88         use_cache: bool = False
89         min_shot_count: int = 1
90
91
92     class AppConfig(BaseModel):
93         """Root configuration object.
94
95         All runtime configuration should be accessed via an instance of this class
96         (or the loader) rather than raw dict lookups.
97         """
98
99         sport: Sport = "badminton"
100        ai_provider: str = "gemini"
101        ai_model: str = "gemini-2.5-flash"
102
103        validation: ValidationConfig = Field(default_factory=ValidationConfig)
104        indexing: IndexingConfig = Field(default_factory=IndexingConfig)
105        output: OutputConfig = Field(default_factory=OutputConfig)
106        stitching: StitchingConfig = Field(default_factory=StitchingConfig)
107
108        # Allow extra keys during transition so existing config.json files do not break.
109        model_config = {
110            "extra": "allow",
111            "validate_assignment": True,
112        }
113
114        # Temporary migration shim for legacy dict-style access (config.get("section",
dict    {}).get(...)).
115        # Will be removed once all call sites (validators, segmenters, etc.) move to typed
116        # attribute access / the typed config. A nested *section* is returned as a plain
dict
117        # via model_dump, so legacy chained access keeps working ? returning the Pydantic
118        # section model instead breaks it (a model has no .get), which bit every validator
on

```

```

119     # real runs. Scalar leaves are returned as-is.
120     def get(self, key: str, default: Any = None) -> Any:
121         _MISSING = object()
122         val: Any = _MISSING
123         if key in type(self).model_fields or hasattr(self, key):
124             val = getattr(self, key, _MISSING)
125         if val is _MISSING:
126             # Legacy convenience: find a leaf key inside any nested section. Sections
are
127             # discovered dynamically from the model fields, so a future top-level
section
128             # (e.g. `audio`, `jobs`) is covered automatically without editing a
hardcoded list.
129             for fname in type(self).model_fields:
130                 section = getattr(self, fname, None)
131                 if isinstance(section, BaseModel) and hasattr(section, key):
132                     val = getattr(section, key)
133                     break
134             if val is _MISSING:
135                 return default
136             if isinstance(val, BaseModel):
137                 return val.model_dump(mode="json")
138             return val
139
140     def __getitem__(self, key: str) -> Any:
141         val = self.get(key)
142         if val is None and key not in type(self).model_fields:
143             # Re-raise like a real dict for unknown top-level
144             raise KeyError(key)
145         return val
146
147
148     # Convenience type alias for callers that want to annotate
149     Config = AppConfig

```

3. user (2026-06-10T06:12:57.667Z)

```

backend\pipeline\segmenters\__init__.py
backend\pipeline\segmenters\gemini.py
backend\pipeline\segmenters\yolo_hybrid.py
backend\pipeline\segmenters\base.py
backend\pipeline\segmenters\motion.py
backend\pipeline\segmenters\tracknet_hybrid.py
backend\pipeline\segmenters\wasb_hybrid.py

```

4. user (2026-06-10T06:13:12.715Z)

```

backend\reporting\spec.yaml:52:      Empty can mean a provider error, exhausted quota, or (for
WASB/TrackNet) a
backend\reporting\spec.yaml:81:      the WASB/TrackNet detector paths). Record with audio
enabled, keep the camera
backend\eval\calibrate_local.py:2:on-device WASB+heuristic knobs to match Gemini's rally labels,
so production runs
backend\eval\calibrate_local.py:6:      WASB trajectory CSV ->
trajectory_to_action_windows(velocity, merge_gap, min_dur)
backend\eval\calibrate_local.py:15:Reuses: TrackNetRunner windowing, rally_gate,
calibrate_wasb.load_golden_gts,
backend\eval\calibrate_local.py:21:(labels = a Gemini full-context CSV; trajectory = the WASB
CSV).
backend\eval\calibrate_local.py:31:from backend.eval.calibrate_wasb import load_golden_gts
backend\eval\calibrate_local.py:40:      """predict_fn(cfg) -> rally windows from a cached WASB
trajectory, LOCAL ONLY
backend\reporting\harvest.py:22:_AI_SEGMENTERS = {"yolo_hybrid", "tracknet_hybrid",
"wasb_hybrid", "gemini"}
backend\eval\distill_local.py:4:further: learn "is this WASB candidate a real rally?" from

```

```

Gemini silver labels,
backend\eval\distill_local.py:9: WASB trajectory -> recall-oriented candidate windows (propose
many; classifier
backend\eval\distill_local.py:18:rally_gate, metrics, calibrate_wasb.load_golden_gts,
gemini_refine.merge_overlaps.
backend\eval\distill_local.py:33:from backend.eval.calibrate_wasb import load_golden_gts
backend\eval\distill_local.py:50:     """WASB trajectory -> (candidate intervals, fps/resolution-
invariant feature rows)."""
backend\eval\export_wasb_segments.py:1:"""Export tuned WASB rally segments for manual eyeball
verification.
backend\eval\export_wasb_segments.py:3:After calibration picks a tuned threshold config, this
turns a cached WASB
backend\eval\export_wasb_segments.py:9:  - calibrate_wasb.make_predict_fn -> trajectory ->
segments for a given config
backend\eval\export_wasb_segments.py:15:    python -m backend.eval.export_wasb_segments
--trajectory ~/clips/sample_long_traj.csv \
backend\eval\export_wasb_segments.py:27:from backend.eval.calibrate_wasb import make_predict_fn,
load_golden_gts, BASELINE
backend\eval\export_wasb_segments.py:140:        out_csv: str =
"output/wasb_tuned_segments.csv", labels_csv: Optional[str] = None,
backend\eval\export_wasb_segments.py:159:    print("=== WASB tuned-segment export ===")
backend\eval\export_wasb_segments.py:199:    ap = argparse.ArgumentParser(description="Export
tuned WASB rally segments for eyeballing.")
backend\eval\export_wasb_segments.py:200:    ap.add_argument("--trajectory", required=True,
help="WASB trajectory CSV (Frame,Visibility,X,Y)")
backend\eval\export_wasb_segments.py:208:    ap.add_argument("--out",
default="output/wasb_tuned_segments.csv")
backend\eval\export_wasb_segments.py:216:    ap.add_argument("--clips-dir",
default="output/wasb_tuned_clips")
backend\eval\calibration.py:20:List[Interval]` (e.g. WASB trajectory ? windows at those
thresholds), so the same
backend\eval\calibration.py:21:harness works for WASB, yolo_hybrid, or anything else. Pure +
unit-tested; no I/O.
backend\eval\calibrate_wasb.py:1:"""Per-video calibration driver: WASB trajectory + golden
rallies -> tuned thresholds + "+X%".
backend\eval\calibrate_wasb.py:4: WASB trajectory CSV (Frame,Visibility,X,Y)
backend\eval\calibrate_wasb.py:8:Run (after a full-video WASB pass exists):
backend\eval\calibrate_wasb.py:9:    python -m backend.eval.calibrate_wasb --trajectory
full_traj.csv \
backend\eval\calibrate_wasb.py:31:BASELINE: Config = (0.02, 2.0, 2.0) # the WasbConfig /
trajectory_to_action_windows defaults
backend\eval\calibrate_wasb.py:53:    """predict_fn(cfg) -> rally windows, from a cached WASB
trajectory (parsed once)."""
backend\eval\calibrate_wasb.py:87:    print("=== WASB rally-segmentation calibration ===")
backend\eval\calibrate_wasb.py:105:    ap.add_argument("--trajectory", required=True, help="WASB
trajectory CSV (Frame,Visibility,X,Y)")
backend\eval\eval_partial_labels.py:16:  - calibrate_wasb.make_predict_fn / load_golden_gts /
BASELINE
backend\eval\eval_partial_labels.py:19:  - export_wasb_segments.build_comparison /
write_comparison_csv (per-rally detail CSV)
backend\eval\eval_partial_labels.py:33:from backend.eval.calibrate_wasb import make_predict_fn,
load_golden_gts, BASELINE
backend\eval\eval_partial_labels.py:35:from backend.eval.export_wasb_segments import
build_comparison, write_comparison_csv
backend\eval\eval_partial_labels.py:192:    ap.add_argument("--trajectory", required=True,
help="WASB trajectory CSV (Frame,Visibility,X,Y)")
backend\eval\gemini_refine.py:1:"""Refine WASB rally candidates with Gemini (verify boundaries +
reject false fires).
backend\eval\gemini_refine.py:3:WASB (shuttle-only) over-fires on between-point service feeds.
This drives the
backend\eval\gemini_refine.py:4:EXISTING Gemini provider over the SHORT WASB candidate clips
(not the whole video):
backend\eval\gemini_refine.py:12:calibrate_wasb.make_predict_fn,
export_wasb_segments.write_segments_csv.
backend\eval\gemini_refine.py:14:This mirrors wasb_hybrid's Phase-3 handoff but runs standalone

```

```

on a cached
backend\eval\gemini_refine.py:15:trajectory (no WASB re-run, no DB) ? an eval/tuning driver, not
the live pipeline.
backend\eval\gemini_refine.py:30:from backend.eval.calibrate_wasb import make_predict_fn,
BASELINE
backend\eval\gemini_refine.py:31:from backend.eval.export_wasb_segments import TUNED,
write_segments_csv, seconds_to_mmss
backend\eval\gemini_refine.py:45:    which over-merged multiple real rallies WASB had bundled
into one candidate.)""
backend\eval\gemini_refine.py:94:    # Keep Gemini's segments separate ? a WASB candidate can
bundle several real rallies
backend\eval\gemini_refine.py:117:    print(f"[gemini_refine] {len(candidates)} WASB candidates
(cfg {cfg}) -> Gemini {model}, "
backend\eval\gemini_refine.py:154:    upload cap), so it finds every rally itself ? not bounded
by WASB's candidate recall.
backend\eval\gemini_refine.py:200:    ap = argparse.ArgumentParser(description="Refine WASB
candidates with Gemini.")
backend\eval\gemini_refine.py:202:    ap.add_argument("--trajectory", default=None, help="WASB
trajectory CSV (required unless --full-video)")
backend\eval\gemini_refine.py:206:                                help="WASB candidate config to feed Gemini
(baseline = whole rallies, good recall)")
backend\eval\gemini_refine.py:217:                                "instead of refining WASB candidates
(needs no --trajectory)")
backend\eval\audio\e1_probe.py:5:WASB windows miss (our recall bottleneck), and (c) form usable
audio-only rally
backend\eval\audio\e1_probe.py:9:Reuses: audio.hit_detector,
calibrate_wasb.load_golden_gts/make_predict_fn,
backend\eval\audio\e1_probe.py:19:from backend.eval.calibrate_wasb import load_golden_gts,
make_predict_fn, BASELINE
backend\eval\audio\e1_probe.py:57:    wasb = make_predict_fn(trajectory, fps,
frame_width)(BASELINE)
backend\eval\audio\e1_probe.py:58:    dur = get_video_duration(video) or (max((e for _, e in gts
+ wasb), default=0.0))
backend\eval\audio\e1_probe.py:62:    f"{len(gts)} labeled rallies, {len(wasb)} WASB
baseline windows")
backend\eval\audio\e1_probe.py:77:    # (2) Recall recovery: of rallies WASB MISSES, how many
have an audio hit-cluster?
backend\eval\audio\e1_probe.py:78:    mr = match_intervals(wasb, gts, iou)
backend\eval\audio\e1_probe.py:82:    print(f" [recall recovery] WASB misses
{len(missed)}/{len(gts)} rallies; "
backend\eval\audio\e1_probe.py:89:    f"F1={sc.f1:.3f} (vs WASB-only F1 {score(wasb, gts,
iou).f1:.3f})")
backend\eval\audio\e1_probe.py:93:    "wasb_missed": len(missed), "recall_recoverable_pct":
100 * rec_rate,
backend\eval\audio\e1_probe.py:94:    "audio_only": sc.as_dict(), "wasb_only_f1":
score(wasb, gts, iou).f1,
backend\eval\audio\e1_probe.py:102:    ap.add_argument("--trajectory", required=True, help="WASB
trajectory CSV")
backend\pipeline\detectors\base.py:1:""Abstract base for detector runners (TrackNet, WASB,
future native, etc.).
backend\pipeline\detectors\base.py:5:the high-accuracy shuttle trajectory substrate used by
tracknet_hybrid / wasb_hybrid.
backend\pipeline\detectors\base.py:17:    (e.g. ".../clip_predict.csv" or ".../clip_wasb.csv"), or
None on any failure.
backend\pipeline\detectors\base.py:64:-
backend\pipeline\segmenters\{tracknet_hybrid,wasb_hybrid}.py (the two callers)
backend\pipeline\segmenters\gemini.py:60:    # A: Check for debug_duration to trim the video
first
backend\pipeline\segmenters\gemini.py:61:    debug_duration = self.config.get("indexing",
{}).get("debug_duration")
backend\pipeline\segmenters\gemini.py:68:    if debug_duration:
backend\pipeline\segmenters\gemini.py:69:        logger.info(f"DEBUG FEATURE: Trimming video
to first {debug_duration} seconds for fast processing...")
backend\pipeline\segmenters\gemini.py:81:    "-t", str(debug_duration),
backend\pipeline\audio\hit_detector.py:8:(ADR-007) if E1 passes. WASB is never modified ? this

```

is a separate signal.

backend\eval\rally_seq_proto.py:3:Motivation (2026-06-08 finding): WASB tracks the shuttle well on Kushagra (97% of GT

[Showing results with pagination = limit: 80]

5. user (2026-06-10T06:13:13.472Z)

```
config.json:38:      "yolo_parallel_chunks": false,
config.json:39:      "yolo_chunk_workers": 2,
README.md:134:      "yolo_parallel_chunks": false,
README.md:135:      "yolo_chunk_workers": 2,
backend\config\models.py:59:    yolo_parallel_chunks: bool = False
backend\config\models.py:60:    yolo_chunk_workers: int = 2
```

6. user (2026-06-10T06:13:22.221Z)

```
1      """WASB shuttle-trajectory runner (WSL2 subprocess adapter).
2
3      Windows-side counterpart to backend/detectors/wasb_infer.py. It:
4          1. stages the video into the WSL filesystem (fast local reads, not /mnt),
5          2. copies wasb_infer.py into the WASB repo's src/ dir,
6          3. runs it in the WSL `wasb` conda env -> a Frame,Visibility,X,Y trajectory CSV,
7          4. reuses the (model-agnostic) trajectory->action-window logic from tracknet_runner.
8
9      So the only WASB-specific bits live here; the windowing + CSV parsing are shared
10     with the TrackNet path. Verified inference core: wasb_infer.py (see its docstring).
11     """
12     import os
13     import shlex
14     import logging
15     import subprocess
16     from dataclasses import dataclass
17     from typing import Dict, Any, Optional, Tuple
18
19     # Reuse the model-agnostic helpers ? trajectory shape + windowing are identical.
20     # parse_trajectory_csv / trajectory_to_action_windows are @staticmethods on
21     TrackNetRunner.
22     from backend.pipeline.detectors.tracknet_runner import to_wsl_mnt_path, TrackNetRunner
23     from backend.pipeline.detectors.base import DetectorRunner
24
25     logger = logging.getLogger(__name__)
26
27     # Windows path to the inference wrapper we stage into WSL.
28     _INFER_WIN = os.path.join(os.path.dirname(os.path.abspath(__file__)), "wasb_infer.py")
29
30     @dataclass
31     class WasbConfig:
32         """Resolved settings for a WASB WSL run (built from config.json ->
33         indexing.wasb)."""
34         distro: str = "Ubuntu"
35         repo_dir: str = "~/models/WASB-SBDT"
36         conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
37         conda_env: str = "wasb"
38         weights_path: str = "~/models/WASB-
39         SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
40         sport: str = "badminton"
41         wsl_stage_dir: str = "~/clips"
42         timeout_sec: int = 0 # 0 = no timeout
43
44     @classmethod
45     def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "WasbConfig":
46         w = (idx_cfg or {}).get("wasb", {}) or {}
47         return cls(
```

```

46         distro=w.get("wsl_distro", "Ubuntu"),
47         repo_dir=w.get("repo_dir", "~/models/WASB-SBDT"),
48         conda_sh=w.get("conda_sh", "~/miniconda3/etc/profile.d/conda.sh"),
49         conda_env=w.get("conda_env", "wasb"),
50         weights_path=w.get("weights_path",
51                             "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"),
52         sport=w.get("sport", "badminton"),
53         wsl_stage_dir=w.get("wsl_stage_dir", "~/clips"),
54         timeout_sec=int(w.get("timeout_sec", 0)),
55     )
56
57
58     class WasbRunner(DetectorRunner): # DetectorRunner ABC (finish item 6 / new item 1)
59         def __init__(self, cfg: WasbConfig):
60             self.cfg = cfg
61
62         def _wsl_bash(self, inner_cmd: str) -> subprocess.CompletedProcess:
63             full = f"source {self.cfg.conda_sh} && {inner_cmd}"
64             argv = ["wsl", "-d", self.cfg.distro, "bash", "-c", full]
65             logger.debug("WSL exec: %s", full)
66             return subprocess.run(argv, capture_output=True, text=True,
67                                 timeout=(self.cfg.timeout_sec or None))
68
69         def healthcheck(self) -> Tuple[bool, str]:
70             """Verify the WSL `wasb` env imports torch and sees a GPU. Returns (ok, msg)."""
71             cmd = (f"conda activate {self.cfg.conda_env} && "
72                  f"python -c \"import torch; print('torch', torch.__version__, "
73                  f"'cuda', torch.cuda.is_available())\"")
74             try:
75                 res = self._wsl_bash(cmd)
76             except FileNotFoundError:
77                 return False, "`wsl` not found ? Windows + WSL2 required."
78             except subprocess.TimeoutExpired:
79                 return False, "WSL healthcheck timed out."
80             out = (res.stdout or "").strip()

```

7. user (2026-06-10T06:13:23.793Z)

```

C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\wasb_runner.py-41-
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\wasb_runner.py-42-         @classmethod
backend\pipeline\detectors\wasb_runner.py-43-         def from_indexing_cfg(cls, idx_cfg: Dict[str,
Any]) -> "WasbConfig":
backend\pipeline\detectors\wasb_runner.py-44:             w = (idx_cfg or {}).get("wasb", {}) or {}
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\wasb_runner.py-45-             return cls(
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\wasb_runner.py-46-                 distro=w.get("wsl_distro",
"Ubuntu"),
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\wasb_runner.py-47-                 repo_dir=w.get("repo_dir",
"~/models/WASB-SBDT"),
--
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\wasb_hybrid.py-86-                 wasb_cfg =
WasbConfig.from_indexing_cfg(idx_cfg)
backend\pipeline\segmenters\wasb_hybrid.py-87-                 runner: DetectorRunner =
WasbRunner(wasb_cfg)
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\wasb_hybrid.py-88-
backend\pipeline\segmenters\wasb_hybrid.py-89:                 w = idx_cfg.get("wasb", {})
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\wasb_hybrid.py-90-                 velocity_thresh =

```



```

w.get("velocity_threshold", 0.02)
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\wasb_hybrid.py-91-           merge_gap =
idx_cfg.get("dead_time_merge_gap", 2.0)
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\wasb_hybrid.py-92-           min_window_duration =
idx_cfg.get("min_action_window_duration", 2.0)
--
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\yolo_hybrid.py-339-           all_candidate_windows = []
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\yolo_hybrid.py-340-
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\yolo_hybrid.py-341-           t_start = time.time()
backend\pipeline\segmenters\yolo_hybrid.py:342:           prefetch_workers =
idx_cfg.get("yolo_prefetch_workers", 2)
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\yolo_hybrid.py-343-
backend\pipeline\segmenters\yolo_hybrid.py-344-           if num_chunks > 1 and prefetch_workers >
0:
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\yolo_hybrid.py-345-           # --- PIPELINED PROCESSING
---
```

8. user (2026-06-10T06:13:37.615Z)

```

backend\config\__init__.py
backend\config\loader.py
backend\config\models.py
```

9. user (2026-06-10T06:13:39.227Z)

```

backend\main.py:30:from backend.config import load_config as load_app_config, AppConfig # new
typed config
backend\main.py:59:global_config: Optional[AppConfig] = None # now typed (Pydantic model)
backend\main.py:62:# New code should import load_app_config / AppConfig from backend.config
directly.
backend\main.py:63:def load_config(config_path: str = "config.json") -> Dict[str, Any]:
backend\main.py:104:     # typed AppConfig: attribute access for the default segmenter (with safe
fallback)
backend\main.py:215:     cfg = load_config() # legacy wrapper still works, returns dict for now
backend\main.py:341:     global_config = load_app_config() # returns typed AppConfig
backend\config\__init__.py:4:     from backend.config import load_config, AppConfig, Config
backend\config\__init__.py:9:from .loader import load_config, save_default_config
backend\config\__init__.py:10:from .models import AppConfig, Config, IndexingConfig,
ValidationConfig # re-export common ones
backend\config\__init__.py:13:     "load_config",
backend\config\__init__.py:15:     "AppConfig",
backend\config\models.py:92:class AppConfig(BaseModel):
backend\config\models.py:149:Config = AppConfig
backend\config\loader.py:3:Provides a single place to load validated AppConfig with built-in
defaults
backend\config\loader.py:6:This replaces scattered json.load + .get() patterns throughout the
codebase.
backend\config\loader.py:16:from .models import AppConfig
backend\config\loader.py:26:     The AppConfig model is the single source of truth for defaults.
backend\config\loader.py:28:     return AppConfig().model_dump(mode="json")
backend\config\loader.py:31:def load_config(path: str | Path | None = None) -> AppConfig:
backend\config\loader.py:38:     Missing file or unreadable file ? returns a fully defaulted
AppConfig.
backend\config\loader.py:47:           file_data = json.load(f)
backend\config\loader.py:56:     return AppConfig.model_validate(merged)
backend\config\loader.py:66:     cfg = AppConfig()
backend\eval\calibrate_local.py:121:           specs = json.load(f)
backend\eval\classifier.py:114:           return cls.from_dict(json.load(f))
```

```

backend\eval\distill_local.py:153:         specs = json.load(f)
backend\infrastructure\database.py:200:         d["stats"] = json.loads(d["stats"] or
"{}")
backend\infrastructure\database.py:201:         d["detection_params"] =
json.loads(d["detection_params"] or "{}")
backend\infrastructure\database.py:210:         d["validation_results"] =
json.loads(d["validation_results"] or "[]")
backend\infrastructure\database.py:250:         r_dict["metadata"] =
json.loads(r_dict["metadata"] or "{}")
backend\infrastructure\database.py:257:         e_dict["details"] =
json.loads(e_dict["details"] or "{}")
backend\eval\regression.py:60:         return json.load(f)
backend\eval\rally_seq_proto.py:205:         run(json.load(f))
backend\providers\gemini.py:124:         segments = json.loads(raw_text)
backend\reporting\__init__.py:42:         # Accept either a plain dict or a typed config object (e.g.
backend.config.AppConfig).
backend\pipeline\detectors\wasb_infer.py:125:         return json.load(f)
backend\pipeline\detectors\wasb_infer.py:171:         obj = json.loads(s)
backend\pipeline\validators\format.py:36:         info = json.loads(result.stdout)

```

10. user (2026-06-10T06:13:45.233Z)

```

1      """Configuration loader and saver.
2
3      Provides a single place to load validated AppConfig with built-in defaults
4      merged with optional file overrides (config.json).
5
6      This replaces scattered json.load + .get() patterns throughout the codebase.
7      """
8
9      from __future__ import annotations
10
11     import json
12     import logging
13     from pathlib import Path
14     from typing import Any
15
16     from .models import AppConfig
17
18     logger = logging.getLogger(__name__)
19
20     DEFAULT_CONFIG_PATH = Path("config.json")
21
22
23     def _get_builtin_defaults() -> dict[str, Any]:
24         """Return a plain dict of the current built-in defaults.
25
26         The AppConfig model is the single source of truth for defaults.
27         """
28         return AppConfig().model_dump(mode="json")
29
30
31     def load_config(path: str | Path | None = None) -> AppConfig:
32         """Load configuration with the following precedence:
33
34         1. Built-in defaults defined in the Pydantic models.
35         2. Values from the JSON file (if present and readable).
36         3. (Future) environment / CLI overrides.
37
38         Missing file or unreadable file ? returns a fully defaulted AppConfig.
39         Unknown keys in the file are tolerated (extra="allow" on the model).
40         """
41         cfg_path = Path(path) if path is not None else DEFAULT_CONFIG_PATH
42
43         file_data: dict[str, Any] = {}

```

```

44     if cfg_path.exists():
45         try:
46             with open(cfg_path, "r", encoding="utf-8") as f:
47                 file_data = json.load(f)
48         except Exception as exc:
49             # Non-fatal: continue with defaults + whatever we could parse.
50             # In production we might log here.
51             logger.warning(f"Failed to read {cfg_path}: {exc}. Using defaults + partial
data.")
52
53     # Merge: start from defaults, overlay file data (Pydantic will validate/coerce).
54     merged = {**_get_builtin_defaults(), **file_data}
55
56     return AppConfig.model_validate(merged)
57
58
59 def save_default_config(path: str | Path | None = None) -> Path:
60     """Write a fresh default config.json.
61
62     Used by setup.py diagnostics and the --setup CLI path.
63     Overwrites if the file exists.
64     """
65     cfg_path = Path(path) if path is not None else DEFAULT_CONFIG_PATH
66     cfg = AppConfig()
67     cfg_path.parent.mkdir(parents=True, exist_ok=True)
68     cfg_path.write_text(cfg.model_dump_json(indent=4), encoding="utf-8")
69     return cfg_path
70
71
72 # Backwards-compatible alias used by some older call sites during transition.
73 def get_default_config() -> dict[str, Any]:
74     """Return defaults as a plain dict (for transitional use only)."""
75     return _get_builtin_defaults()
76

```

11. user (2026-06-10T06:13:46.151Z)

```

55     return "<h3>Sports Highlight Indexer Server is Running. Please create index.html in
backend/api/static/</h3>"
56
57     # Global variables initialized at startup
58     db_instance: Optional[Database] = None
59     global_config: Optional[AppConfig] = None # now typed (Pydantic model)
60
61     # Legacy thin wrapper for any remaining direct calls during transition.
62     # New code should import load_app_config / AppConfig from backend.config directly.
63     def load_config(config_path: str = "config.json") -> Dict[str, Any]:
64         cfg = load_app_config(config_path)
65         return cfg.model_dump(mode="json")
66
67     # --- API Models ---
68     class ProcessRequest(BaseModel):
69         video_path: str
70         player_pool: Optional[List[str]] = None
71         max_frames: Optional[int] = None
72         segmenter: Optional[str] = None
73
74     class QueryRequest(BaseModel):
75         video_id: str
76         server: Optional[str] = None
77         receiver: Optional[str] = None
78         duration_min: Optional[float] = None
79         event_type: Optional[str] = None
80
81     class CompileRequest(BaseModel):

```

```

82         video_id: str
83         segment_ids: List[int]
84         output_filename: str
85
86     # --- API Endpoints ---
87     @app.get("/api/config")
88     def get_config():
89         return global_config
90
91     @app.post("/api/process")
92     def process_video(req: ProcessRequest):
93         if not os.path.exists(req.video_path):
94             raise HTTPException(status_code=404, detail=f"Video file not found at
95 '{req.video_path}'")
96
97         video_id = compute_video_id(req.video_path)
98         was_reingest = db_instance.get_video(video_id) is not None
99
100        # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for the
101        report)
102        logger.info(f"Running validations for {req.video_path}...")
103        val_results, val_context = registry.run_all_with_context(req.video_path,
104        global_config)
105        failures = [r for r in val_results if not r.passed]
106
107        # typed AppConfig: attribute access for the default segmenter (with safe fallback)
108        default_seg = getattr(getattr(global_config, "indexing", None), "default_segmenter",
109        "motion")
110        seg_name = req.segmenter or default_seg
111        segmenter_actual = None
112        segmentation_ran = False
113        response: Optional[Dict[str, Any]] = None
114        try:
115            if failures:
116                db_instance.add_video(video_id, req.video_path, "failed", [r.dict() for r in
117        val_results])
118                response = {
119                    "video_id": video_id,
120                    "status": "failed",
121                    "message": "Video failed validation checks.",
122                    "results": [r.dict() for r in val_results],
123                }
124            return response
125
126        # 2. Run segmenter & indexer
127        print(f"[API] Video passed checks. Segmenting and indexing...")
128        db_instance.add_video(video_id, req.video_path, "processed", [r.dict() for r in
129        val_results])
130
131        segmenter_cls = segmenter_registry.get(seg_name)
132        if not segmenter_cls:
133            available = list(segmenter_registry.list_available().keys())
134            raise HTTPException(
135                status_code=400,
136                detail=f"Segmenter '{seg_name}' not found. Available segmenters:
137        {available}"
138            )
139
140        print(f"[API] Using segmenter: {seg_name}")
141        segmenter_actual = seg_name

```

12. user (2026-06-10T06:14:02.212Z)

```

backend\pipeline\segmenters\gemini.py:39:         idx_cfg = self.config.get("indexing", {})
backend\pipeline\segmenters\gemini.py:61:         debug_duration = self.config.get("indexing",

```

```

{}).get("debug_duration")
backend\pipeline\segmenters\gemini.py:113:         idx_cfg = self.config.get("indexing", {})
backend\pipeline\segmenters\gemini.py:300:         idx_cfg = self.config.get("indexing", {})
backend\pipeline\segmenters\motion.py:43:         idx_cfg = self.config.get("indexing", {})
backend\pipeline\segmenters\tracknet_hybrid.py:69:         idx_cfg = self.config.get("indexing",
{})
backend\pipeline\segmenters\yolo_hybrid.py:75:         idx_cfg = self.config.get("indexing", {})
backend\pipeline\segmenters\yolo_hybrid.py:179:         score_thresh =
self.config.get("indexing", {}).get("detector_score_threshold", 0.5)
backend\pipeline\segmenters\yolo_hybrid.py:258:         min_window_duration =
self.config.get("indexing", {}).get("min_action_window_duration", 2.0)
backend\pipeline\segmenters\yolo_hybrid.py:275:         idx_cfg = self.config.get("indexing", {})
backend\pipeline\segmenters\wasb_hybrid.py:65:         idx_cfg = self.config.get("indexing", {})

```

13. user (2026-06-10T06:14:03.759Z)

```

38:Hybrid segmenter internal 4-phase (yolo_hybrid / tracknet_hybrid / wasb_hybrid ? same shape):
40:P1 chunk video (yolo only; tracknet/wasb = whole-video single pass, chunk_index=0)
48:### 1B. WASB shuttle substrate (P2 of wasb_hybrid)
50:WasbHybridSegmenter.process_video @backend/pipeline/segmenters/wasb_hybrid.py
51: -> WasbRunner.healthcheck() (gate; not ok -> return [])
@backend/pipeline/detectors/wasb_runner.py
52: -> WasbRunner.run_predict(video, out_dir)
53: stage video + wasb_infer.py into WSL fs -> `wsl bash -c 'source conda.sh && python
wasb_infer.py ...'`
54: -> @backend/pipeline/detectors/wasb_infer.py::run (WSL): sliding windows -> GPU
detector pass (CHECKPOINTED det_raw.jsonl) -> CPU tracker (always full re-run) -> trajectory CSV
55: -> TrackNetRunner.parse_trajectory_csv(csv)
@backend/pipeline/detectors/tracknet_runner.py (shared, re-exported by WasbRunner)
58:(`tracknet_hybrid` identical, swapping WasbRunner->TrackNetRunner; `_probe_fps_width`
fallback (30.0, 1280.0).)
62:GT CSV -> annotations.load_annotations / calibrate_wasb.load_golden_gts -> List[Interval]
68:Calibration: calibrate_wasb.run -> calibration.{select_best, improvement_report,
cross_validate} @backend/eval/calibration.py
74:Entrypoints: `@run_eval.py` (single video score), `@run_phase3_eval.py` (manifest batch, can
segment), `@run_regression.py` (baseline gate, exit 1=regression),
`@backend/eval/calibrate_wasb.py` + `@backend/eval/export_wasb_segments.py` (WASB tuning
drivers).
81:- `@backend/main.py` ? Thin CLI + FastAPI bootstrap (orchestrator; modes via FLAGS). Static
UI mounts `/static` from `backend/api/static`. Logging is initialized with `basicConfig` (INFO,
timestamped format); backend modules use `logger = logging.getLogger(__name__)` (replaces stray
prints in hot paths like motion, stitcher, wasb_infer, config loader). User-facing CLI summaries
may still use `print` for direct output.
94: - `::AppConfig` ? root. Scalars `sport='badminton'`, `ai_provider='gemini'`,
`ai_model='gemini-2.5-flash'`; sections `validation`,`indexing`,`output`,`stitching`
(`default_factory`). `model_config = {extra:'allow', validate_assignment:True}`. ?
`extra='allow'` -> unknown/typo'd config.json keys silently kept (legacy tolerance).
109:- `@backend/pipeline/segmenters/__init__.py` ? imports all 5
(`motion,gemini,yolo_hybrid,tracknet_hybrid,wasb_hybrid`) = the de-facto registration manifest;
re-exports them + `segmenter_registry` via `__all__`.
110:[Omitted long matching line]
111:- `@backend/pipeline/segmenters/tracknet_hybrid.py::TrackNetHybridSegmenter` (name
`'tracknet_hybrid'`) ? copy-paste twin of wasb_hybrid; imports `TrackNetRunner`/`TrackNetConfig`
from `pipeline/detectors/tracknet_runner`; `indexing.tracknet.velocity_threshold(0.02)`;
`detection_params` adds `queue_length`. ? any P3/P4 fix must be applied to BOTH (and arguably
yolo_hybrid).
112:[Omitted long matching line]
113:- `@backend/pipeline/segmenters/gemini.py::GeminiPlaySegmenter` (name `'gemini'`) ? pure-AI,
no P2. ? chunks only if `duration>chunk_duration`; `max_workers=5` hardcoded;
`max_segment_duration=90.0` hardcoded; `debug_duration` silently truncates; `parse_time` accepts
colon timestamps (warns); does NOT persist candidate segments (only hybrids feed the fine-tuning
table).
121:[Omitted long matching line]
122:[Omitted long matching line]
123:[Omitted long matching line]

```

```

124:[Omitted long matching line]
130:- `@backend/eval/calibrate_wasb.py` ? WASB tuning CLI. `::BASELINE=(0.02,2.0,2.0)`,
`::default_grid` (45 cfgs), `::make_predict_fn` (parses trajectory once, closes over
`trajectory_to_action_windows`), `::load_golden_gts` ($`start_time,end_time` cols; ? SILENTLY
skips bad rows ? opposite of annotations.load_annotations). ? `{load_golden_gts,
make_predict_fn, BASELINE}` imported by 4 downstream drivers (calibrate_local,
export_wasb_segments, gemini_refine, audio/e1_probe).
132:- `@backend/eval/export_wasb_segments.py` ? tuned cfg -> golden/slicer CSV + comparison +
optional clip cut. `::TUNED=(0.05,1.0,1.0)` (? from ONE video), `::SEG_FIELDS`,`::COMP_FIELDS`,
`::build_comparison`, `::seconds_to_mmss`, `::maybe_slice` (lazy-imports rally_slicer).
`--slice` requires `--video`. ? `write_segments_csv` output loads straight into
`rally_slicer.load_rally_labels` AND re-imported by gemini_refine ? schema is load-bearing.
135:- `@backend/eval/training.py` ? learned-segmenter data layer. `::YOLO_FEATURE_NAMES` (5-d),
`::extract_yolo_features` (? reads `row['stats']` dict from DB; missing fields default 0.0),
`::label_candidates` (same IoU `match_intervals` as evaluator),
`::build_training_set`->(X,y,intervals); `feature_fn` is the substrate plug-in seam.
137:[Omitted long matching line]
138:[Omitted long matching line]
139:[Omitted long matching line]
141:[Omitted long matching line]
145:[Omitted long matching line]
146:- `@backend/eval/audio/e1_probe.py` ? E1 GO/NO-GO diagnostic (no training/fusion).
`::cluster_hits` (groups thwacks; ? `min_hits>=2` so a lone service-feed hit isn't a rally),
`::run` (discrimination ratio + recall recovery + audio-only P/R/F1), `::main`. reuses
`hit_detector` + `calibrate_wasb.{load_golden_gts,make_predict_fn,BASELINE}` + `metrics`.
151:- `@backend/utlis/video.py` ? `::get_video_duration` (ffprobe; ? 0.0 on failure =
"unknown"), `::extract_video_chunk(...,reencode=False,scale_width=None)`. ? copy mode cuts at
nearest keyframe (drift) ? pass `reencode=True` for short/accurate clips; `scale_width` requires
`reencode=True`.
177:[Omitted long matching line]
184:$ **Trajectory CSV** (produced by tracknet/wasb predict, consumed by
`parse_trajectory_csv`):
188:$ **Candidate window dict** (produced by `trajectory_to_action_windows` AND yolo
`_extract_action_windows_from_chunk`):
193:`detection_params` keys by detector:
wasb=`{detector,velocity_thresh,merge_gap,min_action_window_duration,weights_path}`; tracknet
adds `queue_length`; yolo=`{velocity_thresh,merge_gap,frame_skip,tracker,detector_model,detector
_score_threshold,min_action_window_duration}`.
202:- golden/slicer (`calibrate_wasb.load_golden_gts`, `rally_slicer.load_rally_labels`,
`export.SEG_FIELDS`): `rally_number,start_time,end_time[,duration,start_mmss,end_mmss]`;
SILENTLY skips bad rows.
218:## 4. WASB WSL CONTRACT (external dep `~/models/WASB-SBDT`, MIT; NOT in this repo ? claims
below are unverified against external source, only the repo-side caller wasb_infer.py was
checked)
219:- Env: WSL conda `wasb`, cwd `~/models/WASB-SBDT/src`. Hydra `compose(config_name='eval',
overrides=[dataset=<sport>, model=wasb, detector.model_path=<weights>, runner.device=cuda,
runner.gpus=[0]])`. `gpus=[0]` because box is single-GPU. (overrides VERIFIED in
@backend/pipeline/detectors/wasb_infer.py::_build_cfg.)
220:- `model=wasb` -> `name=hrnet`, `frames_in=3`, `frames_out=3`, input 512x288 (WxH), ImageNet
normalize; affine resize maps heatmap coords -> original-frame coords. (model dims read from
cfg["model"] at runtime in wasb_infer.run.)
223:- `dataloaders/dataset_loader.py::ImageDataset(cfg, samples, input_wh, output_wh, transform,
seq_transform, is_train)`; `samples=[{frames:[paths], annos:[{frame_path,
center:Center(is_visible,x,y)}]}]`. (caller constructs exactly this ? VERIFIED in
wasb_infer.run.)
224:- Weights `pretrained_weights/wasb_badminton_best.pth.tar` ? NOT committed, academic-data-
trained -> ? confirm terms before COMMERCIAL deploy. monotrack weights = noncommercial, avoided.
231:- Collector also emits golden rally labels (`rally_number,start_time,end_time,...`) ->
consumed by `@backend/eval/calibrate_wasb.py` / `export_wasb_segments.py` /
`@backend/tools/rally_slicer.py`.
239:- **`AppConfig.get` must return dict sections, not models:** the legacy
`.get("indexing",{ }).get(...)` chains everywhere depend on it; the `extra="allow"` model also
silently keeps typo'd config keys (no rejection).
244:- **fps / frame_width must be PROBED not assumed:** `_probe_fps_width` / runner fallbacks
(30.0, 1280.0) silently mis-scale velocity thresholds; calibrate_wasb CLI bakes `59.94/1920`.

```

The report's `fps\_fallback\_used` flag surfaces this.

```
246:- **ffmpeg NOT in WSL** by design: all ffmpeg/ffprobe run Windows-side (must be on PATH);  
WSL only runs the GPU model. `extract_frames` uses cv2 PNG (slow) ? prefer host ffmpeg +  
`--frames_dir`. ? Don't import `wasb_infer` from Windows code (it imports torch + WASB repo  
modules).  
251:- **Twin segmenters:** `wasb_hybrid.py` / `tracknet_hybrid.py` are copy-paste; fixes to  
P3/P4 must land in both. ? subtle drift: wasb's shot_count `int(None)`-via-TypeError fallback vs  
tracknet's `is None` check.  
253:- **calibrate_wasb.{load_golden_gts, make_predict_fn, BASELINE}` is a de-facto shared API**  
for calibrate_local, export_wasb_segments, gemini_refine, e1_probe ? changing its golden-CSV  
schema or `BASELINE` ripples widely. `gemini_refine.merge_overlaps` reused by `distill_local`;  
`harness.get_predictions` reused by `regression.regress`.
```

[Showing results with pagination = limit: 50]

14. user (2026-06-10T06:14:27.578Z)

```
108 - `::SegmenterRegistry`, `::segmenter_registry` (singleton). ? **register a new  
segmenter**: subclass `BasePlaySegmenter`, call `segmenter_registry.register(MyCls)` at module  
bottom, AND import the module in `@backend/pipeline/segmenters/__init__.py` (registration is an  
IMPORT side-effect). ? `register()` builds `cls(None, {})` just to read `.name` ->  
name/description/`__init__` MUST NOT touch `self.db`/`self.config`. ? `list_available()` re-  
instantiates + swallows exceptions into "No description available." (a throwing `description` is  
masked).  
109 - `@backend/pipeline/segmenters/__init__.py` ? imports all 5  
(`motion,gemini,yolo_hybrid,tracknet_hybrid,wasb_hybrid`) = the de-facto registration manifest;  
re-exports them + `segmenter_registry` via `__all__`.  
110 - `@backend/pipeline/segmenters/wasb_hybrid.py::WasbHybridSegmenter` (name  
`'wasb_hybrid'`) ? WSL WASB substrate (MIT; ships pretrained badminton weights, the LIVE  
default). Imports `WasbRunner`/`WasbConfig` from `pipeline/detectors/wasb_runner`. Config  
`indexing.wasb.velocity_threshold(0.02)` + shared  
`dead_time_merge_gap/min_action_window_duration/ai_handoff_padding/ai_max_workers`. ? whole-  
video single pass; reuses YOLO-named keys `log_yolo_candidates/store_yolo_candidates`; needs  
`{PROVIDER}_API_KEY` (missing -> []); `detection_params` has NO `queue_length` (unlike  
tracknet). ? divergent shot_count: `int(shot_count)` inside try with no `is None` short-circuit  
(copy-paste drift).  
111 - `@backend/pipeline/segmenters/tracknet_hybrid.py::TrackNetHybridSegmenter` (name  
`'tracknet_hybrid'`) ? copy-paste twin of wasb_hybrid; imports `TrackNetRunner`/`TrackNetConfig`  
from `pipeline/detectors/tracknet_runner`; `indexing.tracknet.velocity_threshold(0.02)`;  
`detection_params` adds `queue_length`. ? any P3/P4 fix must be applied to BOTH (and arguably  
yolo_hybrid).  
112 - `@backend/pipeline/segmenters/yolo_hybrid.py::HybridYoloSegmenter` (name  
`'yolo_hybrid'`, no YOLO remains ? torchvision detector + supervision ByteTrack; ADR-005).  
`::PERSON_CLASS_ID=1` ? (ultralytics used 0; wrong value -> zero detections).  
`::lazy_load_model` (lazy `torchvision` import so module/tests load without it),  
`::extract_action_windows_from_chunk`, `::make_tracker` (per-chunk ByteTrack ? track IDs  
chunk-local), `::DETECTOR_FACTORIES`/`::DEFAULT_DETECTOR='fasterrcnn_resnet50_fpn'`. ?  
velocity at `effective_fps = fps/frame_skip` -> `yolo_velocity_threshold` coupled to  
`frame_skip`; pipelined-vs-sequential P2 (prefetch ffmpeg overlaps sequential GPU);  
`yolo_tracker`/`bytetrack.yaml` is dead/back-compat. ? `gemini_*` config keys are deprecated  
aliases still honored.  
113 - `@backend/pipeline/segmenters/gemini.py::GeminiPlaySegmenter` (name `'gemini'`) ?  
pure-AI, no P2. ? chunks only if `duration>chunk_duration`; `max_workers=5` hardcoded;  
`max_segment_duration=90.0` hardcoded; `debug_duration` silently truncates; `parse_time` accepts  
colon timestamps (warns); does NOT persist candidate segments (only hybrids feed the fine-tuning  
table).
```

15. user (2026-06-10T06:14:27.737Z)

```
1 {  
2   "sport": "badminton",  
3   "ai_provider": "gemini",  
4   "ai_model": "gemini-2.5-flash",  
5   "validation": {  
6     "min_resolution_width": 1280,
```

```

7         "min_resolution_height": 720,
8         "min_fps": 29.9,
9         "min_blur_threshold": 20.0,
10        "max_scene_cuts_per_minute": 0.5,
11        "relevance": {
12            "court_green_lower": [
13                35,
14                40,
15                40
16            ],
17            "court_green_upper": [
18                85,
19                255,
20                255
21            ],
22            "court_pixels_min_ratio": 0.05
23        }
24    },
25    "indexing": {
26        "default_segmenter": "gemini",
27        "use_gpu": true,
28        "motion_threshold": 0.08,
29        "min_segment_duration": 3.0,
30        "dead_time_merge_gap": 2.0,
31        "chunk_duration": 90,
32        "chunk_overlap": 10,
33        "detector_model": "fasterrcnn_resnet50_fpn",
34        "detector_score_threshold": 0.5,
35        "yolo_velocity_threshold": 0.15,
36        "yolo_frame_skip": 3,
37        "yolo_tracker": "bytetrack.yaml",
38        "yolo_parallel_chunks": false,
39        "yolo_chunk_workers": 2,
40        "min_action_window_duration": 2.0,
41        "log_yolo_candidates": true,
42        "store_yolo_candidates": true,
43        "ai_handoff_padding": 2.0,
44        "ai_max_workers": 5,
45        "tracknet": {
46            "wsl_distro": "Ubuntu",
47            "repo_dir": "~/models/TrackNetV4",
48            "conda_sh": "~/miniconda3/etc/profile.d/conda.sh",
49            "conda_env": "TrackNetV4",
50            "weights_path": "",
51            "queue_length": 5,
52            "stage_in_wsl": true,
53            "wsl_stage_dir": "~/clips",
54            "timeout_sec": 0,
55            "velocity_threshold": 0.02
56        }
57    },
58    "output": {
59        "default_highlights_name": "highlights.mp4",
60        "db_name": "sports_indexer.db"
61    },
62    "stitching": {
63        "begin_padding": 0.5,
64        "end_padding": 0.5,
65        "use_cache": false,
66        "min_shot_count": 1
67    }
68 }

```


4:"config & identity consistency" tech-debt pass. Each item below becomes its own PR
9:remaining high-value hygiene items not already done (config/structure/Result/logging/
33: - `gpu` ? torch/torchvision (note: the cu124 `--extra-index-url` is ****not****
46: `--extra-index-url` install path (the current `setup.py` logic can move into docs).

17. user (2026-06-10T06:14:41.754Z)

```
133-         print(f"[API] Using segmenter: {seg_name}")
134-         segmenter_actual = seg_name
135-         segmenter = segmenter_cls(db_instance, global_config)
136-         result = segmenter.process_video(video_id, req.video_path, req.player_pool,
req.max_frames)
137-         segmentation_ran = True
--
393-         print(f"[CLI] Indexing play segments using segmenter: {seg_name}...")
394-         segmenter_actual = seg_name
395-         segmenter = segmenter_cls(db_instance, global_config)
396-         result = segmenter.process_video(video_id, args.video_path,
max_frames=args.max_frames)
397-         segmentation_ran = True
```

18. user (2026-06-10T06:14:45.793Z)

```
type: dict
wasb present: False
yolo_prefetch_workers present: False
debug_duration present: False
default_segmenter (declared key honored): wasb_hybrid
top-level extra kept: 1
pydantic 2.13.4
```

19. user (2026-06-10T06:15:28.339Z)

Structured output provided successfully

20. assistant (2026-06-10T06:15:33.697Z)

Verification complete ? finding confirmed as real with high confidence.